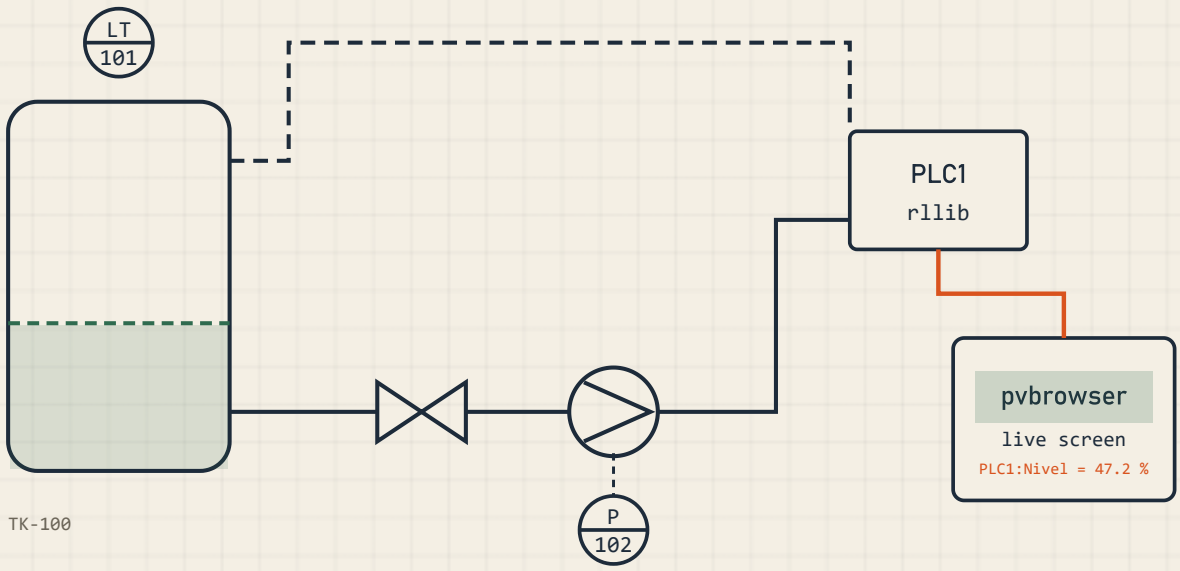
OPEN SCADA / HMI

PV Suite

FREE SOFTWARE
BUILT FOR THE PLANT

Supervisory screens you lay out in the Designer, wire to the PLC without detours, and run on Windows, Linux and macOS.



from the sensor to the screen, no code written

VERSION
1.0.0-beta.4

PLATFORMS
Windows · Linux · macOS

LICENSE
GPL / LGPL

What PV Suite is

PV Suite takes pvbrowser, a free SCADA with many years on the shop floor, moves it to Qt 6 and adds what used to take the longest by hand: tags, the communication daemon and the logic behind every button.

The goal is simple: a supervisory screen you can build with the mouse, a single place where the link to the PLC is declared, and your own code still there when something finer is needed.

It does not replace the programmer; it takes the repetitive part off their hands. What used to be hundreds of lines to read a register, scale it and paint it a colour is now one row in a table.

WHO IT IS FOR

- Integrators building panels and supervision for small and medium customers.
- Maintenance technicians who want to see the plant without expensive licences.
- Teachers and students of automation who need something real to practise with.
- Anyone with pvbrowser projects who wants to bring them to Qt 6 without starting over.

> *“A screen should show what happens in the plant, not how hard it was to program.”*

The four pieces

PB

PVBROWSER

The client: displays the screens sent by the server. Includes a web browser.

PS

PVSERVER

The library used to program the screen servers, one per project.

PD

PVDEVELOP

The workbench: editor, screen Designer, tags and daemon generator.

RL

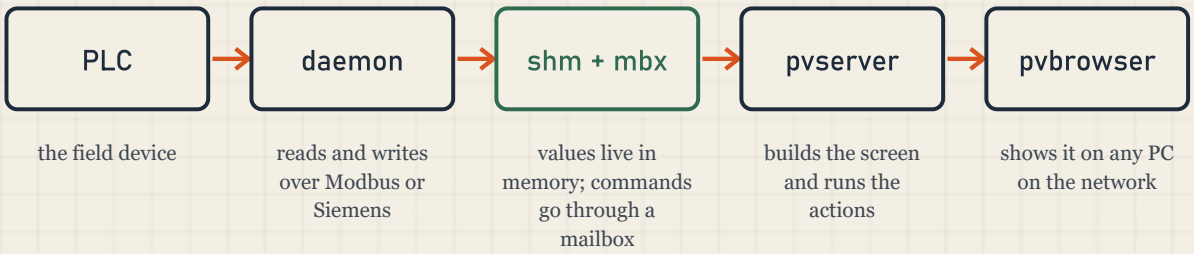
RLLIB

Communication: Modbus, Siemens, shared memory, mailboxes and tags.

02 · INSIDE

How a value travels

Each piece does one thing and does it well. A communication fault does not bring the screens down, and a frozen screen does not stop the PLC from being read.



The daemon is generated by pvdevelop from the tag table. It polls the PLC, stores every value with its quality in shared memory and handles the writes that arrive through the mailbox.

The pvserver never talks to the PLC: it reads that memory. If the daemon stops, the screen knows and flags the value as bad instead of showing an old number as if it were fresh.

pvbrowser is lightweight: it receives the screen over the network and sends the clicks back. Several can be open at once, from different PCs.

NO CODE REQUIRED

In the Properties of every widget, the **Tags...** button defines what it does and how it looks. pvdevelop generates the code when the screen is saved. Hovering over each field shows what it is for and which widgets use it.

ACTIONS

write a value, toggle, add or subtract within limits, enter a number, go to another screen, ask for confirmation, act only when a condition holds.

ANIMATIONS

formatted text, colour by rules, blinking, visibility, image by value, image sequences and a bad-data marker.

```
//pvb: btnBomba press=write:PLC1:Bomba=1
  release=write:PLC1:Bomba=0
color=PLC1:Bomba
```

What you get

NAMED TAGS

Structures, arrays and bits of integers (`PLC1:Estado.3`). Defined once, used on every screen.

SIMULATION

The daemon starts with `-sim` against a Modbus simulator: test everything without the PLC at hand.

TAG MONITOR

Live values with quality. Write and force them while commissioning.

A FRIENDLY DESIGNER

Copy, paste, align and distribute widgets; a double click opens Properties.

BUILT-IN COMPILING

A panel with errors marked in the editor. MinGW or Visual Studio on Windows, g++ on Linux, with each project's Qt chosen from the toolbar.

FOUR LANGUAGES

pvdevelop and pvbrowser in Spanish, English, Portuguese and German, with manual and built-in help.

First steps

1. In pvdevelop, **File > New**: folder and project name.
2. **Daemon > Modbus**: PLC address and the tag table. Compiling generates the daemon and its `.tagmap` .
3. In the Designer, insert widgets and use **Tags...** in Properties to give them actions and animations.
4. **Ctrl+T** builds and runs the server; **Ctrl+P** opens pvbrowser to see it.
5. **Daemon > Tag monitor** to watch and force values. The `01_modbus` example has it all set up.

Installation

Windows 64-bit: one installer that puts everything in `C:\PVSuite`. If you only need to watch, pvbrowser can be installed alone.

Linux x86-64: a `.run` file that installs into `/opt/pvb` with Qt included. If the desktop lacks a library, `preparar_linux.sh` installs it.

macOS (Apple Silicon): pvbrowser and pvdevelop in a zip, still in beta and unsigned.

If it is already installed, the installer offers to update it: nothing needs to be deleted by hand.

LICENSE

pvbrowser and pvdevelop are GPL. pvserver and rllib are LGPL, so the servers you build can carry any licence you choose.

OLDER PROJECTS

pvbrowser projects keep working. With the MinGW 8.1 component they still build as before, 32 or 64-bit, and can be moved to Qt 6 at your own pace.

A few words

PV Suite was born from using pvbrowser for real work and wanting the tedious parts to stop being tedious. Every feature comes from a concrete need: a button that had to write a register, a screen that did not tell you the daemon was down.

It is free software and still in beta: try it, break it, improve it. If something does not behave as you expect, say so on GitHub; that is the best way for the next version to come out better.

Néstor Palomeque

WHERE TO FIND IT

SOURCE CODE
`github.com/nestorlevi/PvSuite`

INSTALLERS
`github.com/nestorlevi/PvSuite/releases`

SUPPORT THE PROJECT
`cafecito.app/nestorlevi`
If you find it useful, you can buy me a coffee (Cafecito, Argentina).

BASED ON
`pvbrowser.de` — Rainer Lehrig